FIAP · GLOBAL SOLUTION 2026 · DEVOPS & DEVSECOPS CHALLENGE

Queimadas API
Pipeline DevSecOps de CI/CD

CitizenTech: da cidade ao campo, tecnologia para a vida cotidiana

Participante	**Luiz Gustavo Gonçalves Brito**
RM	562192
Turma	2TCNPZ
Modalidade	Individual · Opção A (projeto próprio)
Data	12 de novembro de 2026

GitHub · Jenkins · SonarQube · Trivy · Docker · GHCR · Docker Compose · Terraform · Azure

2 Introdução e relação com CitizenTech

As queimadas afetam diretamente a qualidade do ar, a saúde e a economia de quem vive nas cidades e no campo. Dados abertos do INPE mostram a evolução dos focos por bioma, mas raramente chegam ao cidadão em um formato simples de consultar.

A **Queimadas API** expõe esses dados (focos, intensidade média do fogo, precipitação e dias sem chuva por bioma e mês, 2024–2025) por meio de uma API HTTP. O foco deste trabalho, porém, é a forma como ela é entregue: uma pipeline DevSecOps que testa, analisa a qualidade, verifica a segurança da imagem, publica e faz o deploy automaticamente a cada alteração no repositório. Assim, o dado público chega ao cidadão de forma **confiável, segura e contínua**.

3 Objetivos e descrição da aplicação

Objetivos

- Automatizar o ciclo completo GitHub → Jenkins → Testes → SonarQube → Docker Build → Trivy → Registry → Deploy → Validação.
- Bloquear automaticamente entregas com falha de teste, Quality Gate reprovado ou vulnerabilidade CRITICAL.
- Provisionar e configurar todo o ambiente como código (Terraform, JCasC, Docker Compose), sem passos manuais na UI.

Aplicação

Item	Descrição
Stack	Python + FastAPI, servido por Uvicorn
Endpoints	`GET /health` (status) · `GET /queimadas?bioma=Pantanal` (focos por bioma e mês, filtro opcional)
Testes	`pytest` + `httpx` (TestClient), cobertura com `pytest-cov`
Imagem	Dockerfile multi-stage com `cgr.dev/chainguard/python` (Wolfi): estágio `-dev` instala as dependências num venv; o estágio final é distroless, sem shell nem gerenciador de pacotes
Repositório	https://github.com/luizbrito7/gs02-devops.fiap

4 Ambiente e arquitetura

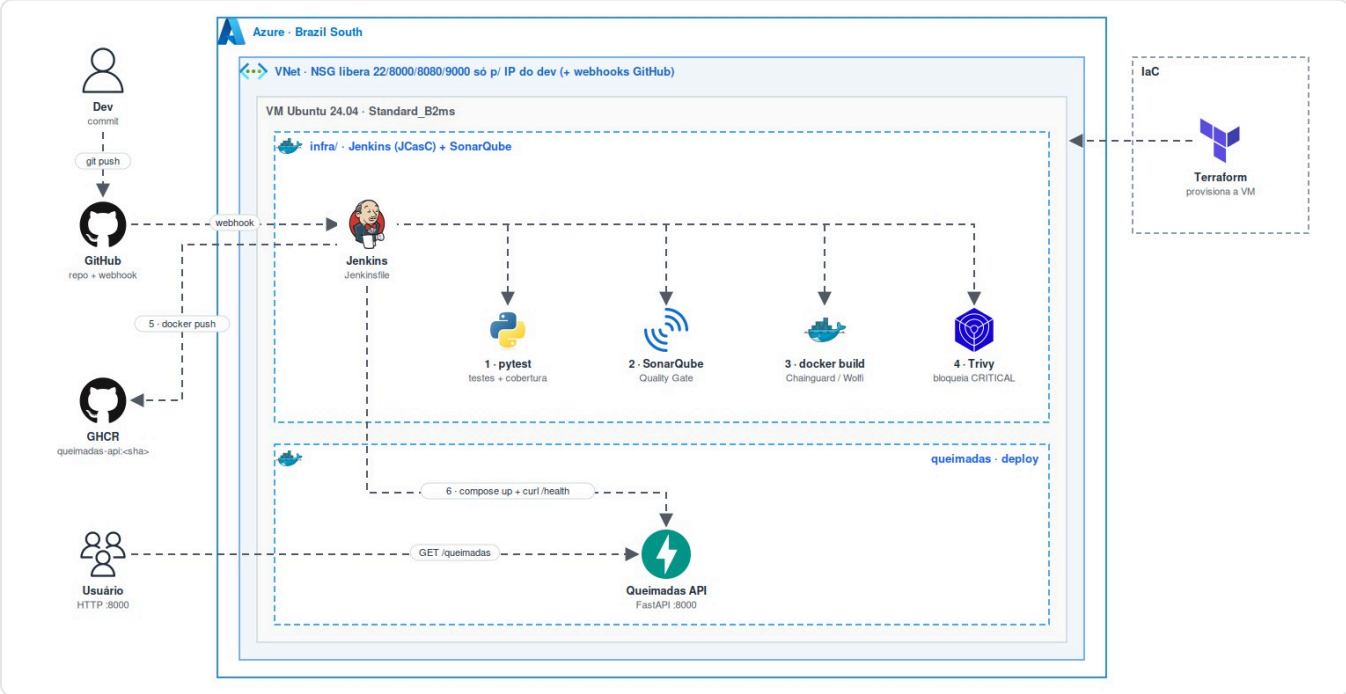


Figura 1: arquitetura. Push no GitHub dispara o Jenkins por webhook; as etapas rodam como containers na VM; a imagem vai para o GHCR e é implantada com Docker Compose.

Camada	Implementação
Nuvem / VM	Azure (Brazil South), VM Ubuntu 24.04 `Standard_B2ms` (2 vCPU, 8 GB, 64 GB), provisionada com **Terraform** (`terraform/`). O cloud-init instala Docker e o sysctl exigido pelo SonarQube.
Rede	VNet + NSG: portas 22, 8000, 8080 e 9000 liberadas apenas para o IP do desenvolvedor; 8080 também para as faixas de webhook do GitHub.
CI/CD	**Jenkins** LTS (JDK 21) com Docker CLI, configurado 100% via **Configuration as Code** (`infra/jenkins.yaml`): usuário admin, credenciais, servidor Sonar e o job (Job DSL) com trigger por push.
Qualidade	**SonarQube** Community; `infra/setup.sh` configura via Web API a senha do admin, o token de análise e o webhook do Quality Gate.
Segurança	**Trivy** 0.76.2 (versão fixa) analisando a imagem construída.
Registry	**GitHub Container Registry** (`ghcr.io/luizbrito7/queimadas-api`), registry equivalente ao Docker Hub.
Deploy	**Docker Compose** na mesma VM, porta 8000.

Segredos (senhas, token do Sonar e PAT do GHCR) ficam somente no `infra/.env` da VM, fora do Git; o repositório traz apenas o modelo `.env.example` .

5 Etapas implementadas

Etapa do enunciado	Como foi atendida
1 · Ambiente e repositório	Repositório público com código, Dockerfile, Jenkinsfile, Compose, testes e README. Imagem construída e executada em container.
2 · Pipeline Jenkins	`Jenkinsfile` declarativo com checkout, testes, SonarQube, Quality Gate, build, Trivy, push, deploy e validação. Disparo automático por **webhook do GitHub** a cada push na `main`.
3 · SonarQube	Análise com `sonar-scanner-cli` dentro de `withSonarQubeEnv`; `waitForQualityGate abortPipeline: true` bloqueia a pipeline se o gate reprovar.
4 · Trivy	Duas execuções: relatório HIGH/CRITICAL arquivado (`trivy-report.txt`) e bloqueio com `--severity CRITICAL --exit-code 1`. Nenhum controle foi desabilitado.
5 · Publicação e deploy	Tag com o SHA curto do commit + `latest`, push no GHCR, `docker compose up` e validação HTTP com `curl` em `/health` e `/queimadas`.
6 · Troubleshooting	Falha controlada de segurança (seção 8).

Trecho do Jenkinsfile: Quality Gate e bloqueio do Trivy

```
stage('Quality Gate') {
  steps { timeout(time: 5, unit: 'MINUTES') { waitForQualityGate abortPipeline: true } }
}
stage('Trivy') {
  steps { script {
    sh "${trivy} --severity HIGH,CRITICAL --format table -o trivy-report.txt $IMAGE:$TAG"
    sh "${trivy} --severity CRITICAL --exit-code 1 $IMAGE:$TAG"   // bloqueio
  } }
}
```

6 Resultados SonarQube e Trivy

SonarQube

Quality Gate	Cobertura	Bugs	Vulnerabilidades	Code smells	Security hotspots	Duplicação	Ratings
Passed	100%	0	0	0	0	0%	A / A / A

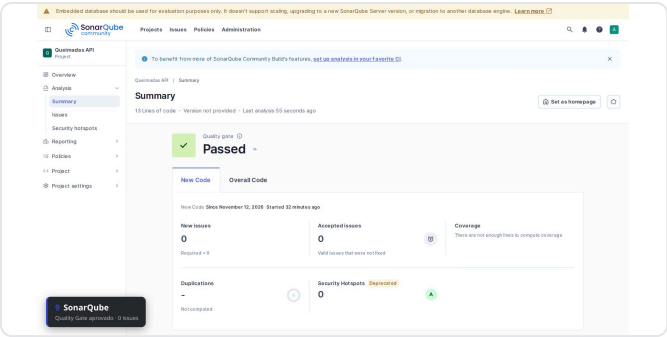


Figura 2: dashboard do SonarQube, Quality Gate aprovado.

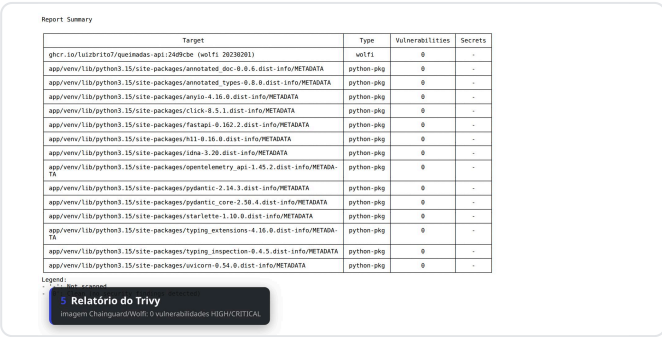


Figura 3: relatório do Trivy arquivado pelo Jenkins.

Trivy

A imagem final (Chainguard/Wolfi) e todos os pacotes Python do venv foram analisados: **0 vulnerabilidades HIGH ou CRITICAL**. A escolha da base distroless foi decisiva: elimina shell, gerenciador de pacotes e bibliotecas de sistema desnecessárias.

7 Evidências da pipeline e do deploy

Build	Gatilho	Resultado	Observação
#1–#4	webhook / manual	FAILURE	montagem da pipeline (ver seção 8)
#5	manual	SUCCESS	primeira execução de ponta a ponta
#6	webhook	FAILURE	falha controlada, bloqueada pelo Trivy
#7–#12	webhook / manual	SUCCESS	correção e execuções seguintes (~1 min cada)



Figura 4: Stage View com o histórico: build #6 bloqueado no Trivy e builds seguintes verdes.

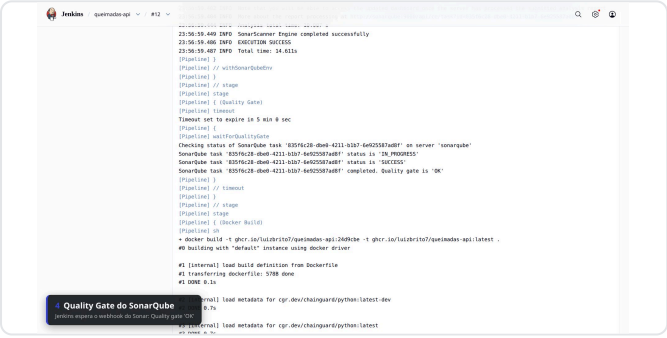


Figura 5: log, Quality gate is 'OK'.

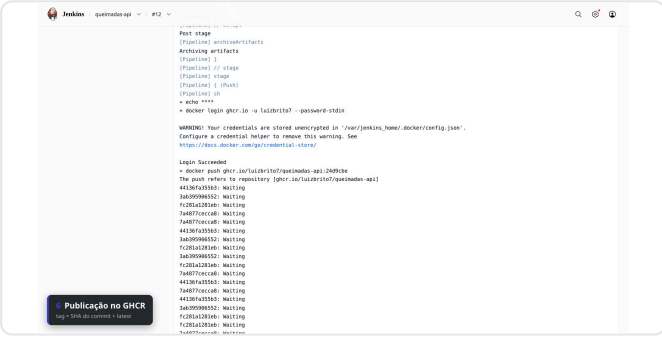


Figura 6: push da imagem no GHCR (tag = SHA).

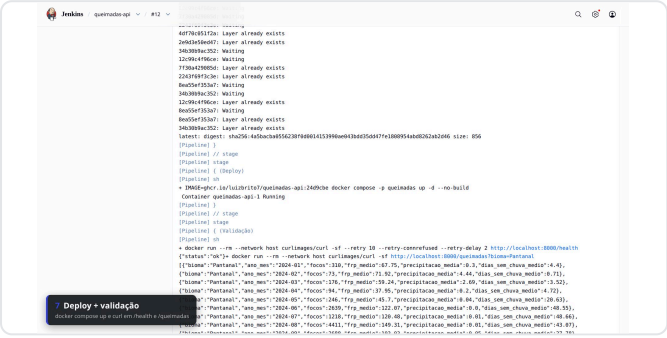


Figura 7: deploy com Compose e validação via curl.

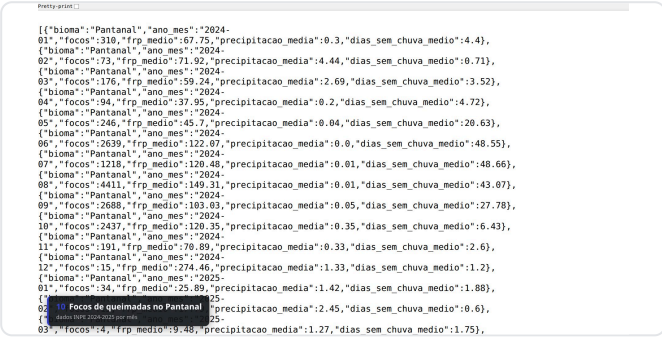


Figura 8: API no ar, /queimadas?bioma=Pantanal.

8 Troubleshooting

Falha controlada (etapa 6): dependência com CVE CRITICAL

Build	Commit	Resultado
#6	`65f05db` : adiciona `Django==3.2.13`	**Trivy: Total 3 (CRITICAL: 3), incluindo CVE-2022-34265 (SQL injection em Trunc/Extract). Pipeline interrompida antes do push e do deploy.**
#7	`18d3f82` : remove a dependência vulnerável	**Todas as etapas verdes, 0 vulnerabilidades.**

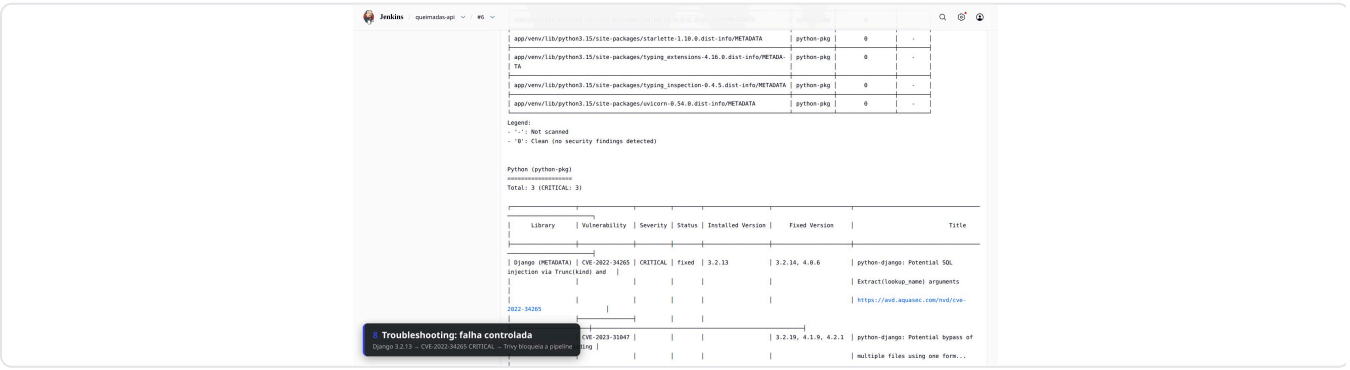


Figura 9: log do build #6, Trivy identificando 3 CVEs CRITICAL no Django 3.2.13.

Análise: a severidade foi confirmada na base GHSA (GHSA-p64x-8rxx-wf6q, *critical*, corrigida na 3.2.14). O bloqueio ocorreu na segunda execução do Trivy (`--exit-code 1`); como o deploy depende das etapas anteriores, a versão vulnerável nunca chegou ao registry nem ao ambiente.

Falhas reais encontradas na montagem

Problema	Causa	Correção
`ModuleNotFoundError: app` nos testes	`pytest` chamado direto não inclui o diretório atual no path	`python -m pytest`
Quality Gate: *Unable to guess SonarQube task id*	a imagem do scanner grava `report-task.txt` fora do workspace	`-Dsonar.working.directory=$WORKSPACE/.scannerwork`
Webhook do Sonar recusado	o SonarQube bloqueia URLs de IP privado (rede Docker interna)	`sonar.validateWebhooks=false` via API, no `setup.sh`
Push: *password is empty*	credencial do registry ainda não preenchida	PAT (classic, `write:packages`) no `.env` , lido pelo JCasC

9 Conclusão

A pipeline cumpre o fluxo exigido de ponta a ponta e bloqueia de forma automática e verificável os três tipos de falha (teste, qualidade e segurança). Os principais aprendizados foram:

- **Segurança começa na imagem base:** com Chainguard/Wolfi o Trivy encontrou zero vulnerabilidades, e o único CRITICAL veio de uma dependência da aplicação, exatamente o tipo de risco que o gate precisa barrar.
- **Tudo como código:** Terraform, JCasC/Job DSL, Compose e o script do Sonar permitem destruir e recriar o ambiente inteiro sem cliques na UI.
- **Ferramentas em containers** exigem atenção a caminhos: montar o workspace no mesmo caminho do host resolveu a cobertura e o `report-task.txt`.
- **Ler os logs antes de mudar o código:** cada falha real da montagem foi resolvida a partir da mensagem exata do log.

Como evolução, a imagem poderia ser assinada (cosign) e o deploy separado em ambientes (staging/produção) com aprovação manual.

10 Links

Entregável	Link
GitHub	https://github.com/luizbrito7/gs02-devops.fiap
Vídeo	https://youtu.be/Oe6oRs7hoRA
Imagem	ghcr.io/luizbrito7/queimadas-api